
AlphaGo Zero: A Self-Play Reinforcement Learning Algorithm in the Game of Go

CHAN, Cheuk Man WONG, Hok Fong
Department of Computer Science and Engineering
The Chinese University of Hong Kong
{1155194886, silaswhf}@link.cuhk.edu.hk

Abstract

In this report, we discuss the ground-breaking advancement in the realm of game-playing AI, AlphaGo Zero, developed by Google DeepMind [1]. By combining the Monte-Carlo tree-based search and a non-linear deep neural network position evaluation, a self-reinforcing feedback loop is formed, enabling a tabula rasa, rapidly improving learning scheme without any expert data sets. The architecture, training methodology, and analyses will be highlighted to discuss AlphaGo Zero's superhuman performance in the game of Go, which can be considered as the most challenging domains in terms of human intellect. Furthermore, we will discuss the implications of AlphaGo Zero's success on the subsequent development of training algorithms for game-playing agents that are applied beyond board games.

1 AI agents in the game of Go

With more than 10^{170} possibilities of legal board configurations, the huge search space in the game of Go prevents any clever heuristics from existing, as moves in Go have profound consequences far beyond simple evaluation in the future of the game. For more than four decades the most advanced chess programs have employed alpha-beta search methods with handcrafted evaluation functions. On the other hand, traditional MCTS were much weaker than alpha-beta search programs. AlphaGo Zero effectively captures the advantages of MCTS and neural networks, cleverly diminishing the worse-case approximation errors induced through averaging over position evaluations within a subtree.

In recent years, improvements on the efficiency of computational power and optimization algorithms have enabled breakthroughs in the development of AI agents that surpassed human intelligence in complex domains. We have seen an expansion in domain, unlimited to zero-sum two-player games, and minimized importance on prior knowledge of game-playing agents, through a combination of deep neural networks with advanced search algorithms.

2 The AlphaGo Zero algorithm

2.1 Formalization

A reinforcement learning algorithm decides a move in each step of interaction, and receives an observation from the environment. The reinforcement learning problem can be modeled as a Markov Decision Process (MDP), described by a 5-tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{U}, \gamma \rangle$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of actions, $\mathcal{T} : \mathcal{S} \times \mathcal{A} \rightarrow p(\mathcal{S})$ is the transition dynamics function, with $\mathbb{P}(s'|s, a)$ being the probability of transitioning into state s' , starting from state s and taking action a , $\mathcal{U} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function mapping each action for each state to a real number, and $\gamma \in [0, 1]$ is the discount factor indicating the importance of reward for the agent in the future. In the game of Go, rewards only

appear apparent at the end of the game, and discount factor is set to 1 as no restriction on the length of the game is imposed. Hence, the AlphaGo Zero algorithm considers two questions: accurately model the value function for each state, and model the policy distribution π (as a strategic principle) to maximize the cumulative reward, given a starting state s , i.e.

$$V(s) = \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t u_t | s_0 = s \right].$$

2.2 An overview of the self-reinforcement learning scheme

AlphaGo Zero incorporates a deep neural network $f_{\theta}(s) = (\mathbf{p}, v)$, taking a raw board representation s of the position and its history, and outputs two items - a probability distribution and an estimated probability of winning from position s . The vector of move probabilities $\mathbf{p}$ corresponds to the probability of selecting each move a , $p_a = \mathbb{P}(a|s)$. Then, the neural network in AlphaGo Zero is trained from games of self-play by Monte-Carlo Tree Search (MCTS) algorithm. In each position s , an MCTS search is carried out, guided by the neural network f_{θ} . Then, the probabilities generated by the MCTS search are able to demonstrate much powerful moves than the raw probabilities provided by the neural network $f_{\theta}(s)$. This provides a policy evaluation measurement, leading to updates of the neural network's parameters, where the move probabilities and value $(\mathbf{p}, v)$ are made to match more closely to the improved search probabilities and self-play winner (π, z) .

2.3 The Monte-Carlo Tree Search algorithm

MCTS works by iteratively building a search tree from the current state, selecting promising nodes to explore, expanding the tree with new nodes, simulating game outcomes from these new nodes using Monte Carlo sampling, and updating the tree structure with the results. One major advantage of MCTS is that it requires little or no domain knowledge to conduct the search, making it a popular choice for planning in complex domains, such as game of Go.

2.3.1 Simulation

The general MCTS process involves four phases, namely selection, expansion, simulation and backup. However, AlphaGo Zero uses a much simpler MCTS search algorithm (shown in Figure 1), which does not require any simulation (rollout) during the search. Instead, evaluation of the position is done by the neural network. Every node s in the search tree represents a specific board configuration, storing the set of statistics $\{N(s, a), Q(s, a), P(s, a)\}$ corresponding to the visit count, the average state-action value, and the prior probability of selecting action a in s . These are the three phases of MCTS in AlphaGo Zero:

- **Selection:** Starting from the root node, actions are recursively selected using the PUCT bound to select the most promising child node, until the process reaches an expandable node.
- **Expansion and Evaluation:** When the search process reaches a leaf node s_L in the search tree, the search tree is expanded by adding one or more child nodes to it, according to the available moves (or actions). The game state s_t corresponding to the leaf node is used as the input to the neural network f_{θ} to obtain the corresponding output $(\mathbf{p}, v)$, which is used to initialize the child nodes by setting $N(s, a) = 0$, $Q(s, a) = 0$, $P(s, a) = p_a$. Then we back to selection stage starting from the leaf node s_L .
- **Backup:** The process of selection and expansion are done alternatively until we reach the maximum depth. The results from the simulation are backed up to the traversed path from the leaf node s_L to the root node, and the statistics are updated for each time step $t \leq L$. Along the trajectory, the visit count $N(s_t, a_t)$ is updated as $N(s_t, a_t) = N(s_t, a_t) + 1$, and the mean state-action value $Q(s_t, a_t)$ is updated as $Q(s_t, a_t) = (N(s_t, a_t)Q(s_t, a_t) + v)/(N(s_t, a_t) + 1)$ where v is the value output of the neural network $f_{\theta}(s_L)$.

Specifically, in the selection process, an action a_t is selected to maximize the Probabilistic Upper Confidence Tree (PUCT) Bound, constructed by the sum of average state-action value (known as the

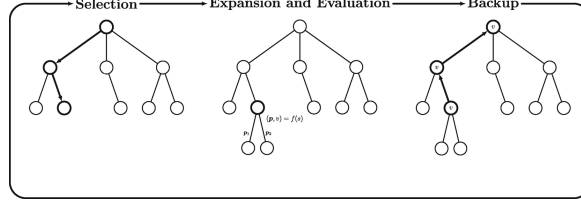


Figure 1: MCTS algorithm for AlphaGo Zero.

exploitation term) $Q(s_t, a)$ and an exploration term $U(s_t, a)$, leading to the following selection rule:

$$a_t = \arg \max_a \left\{ Q(s_t, a) + \underbrace{c_{\text{puct}} p(s_t, a) \sqrt{\frac{\sum_{b \in \mathcal{A}} N(s_t, b)}{1 + N(s_t, a)}}}_{U(s_t, a)} \right\}$$

This selection scheme balances exploitation and exploration, allowing better values by exploring the unknown, sub-optimal valued states. The less the action is attempted, the greater the exploration term will be. Eventually, the simulation would favor actions with high action value with enough number of simulations. In the expansion process, with the perfect knowledge of the game rules given, it is unnecessary to create nodes that represent illegal moves. Hence, in practice, we would be able to speed up the expansion process by restricting the creation of nodes when performing the large amount of iterations. We also note that in zero-sum two-player games, the value v that is being backed up is occasionally flipped, since the opponent's utility should be negated if the current player is the maximizing player.

2.3.2 Self-play

AlphaGo Zero is able to generate high-quality data with massive number of self-play, consisting of 25,000 games. For each time step $0 \leq t < L$, the algorithm conducts 1,600 simulations of MCTS using the current set of best network parameters. After the search is completed, a stronger search policy π_t is computed for the current game state s_t . A move from π is then selected for the current player. This policy π_t is proportional to the exponential visit count of the children based on the root node, with a temperature parameter τ controlling the level of exploration, leading to the distribution

$$\pi(a|s_0) = \frac{N(s_0, a)^{1/\tau}}{\sum_b N(s_0, b)^{1/\tau}}.$$

For the first 30 moves of each self-play game, the temperature in the softmax function (defined as $\text{softmax}(x_i) = \exp(x_i/\tau) / \sum_j \exp(x_j/\tau)$) is set to $\tau = 1$. This selects moves proportionally to the visit count of the children in the root node of the MCTS search tree. For the remainder of the game, a temperature close to zero is used ($\tau \rightarrow 0$), which corresponds to always selecting the most visited child of the root node of the MCTS search tree.

To encourage further exploration, a small amount of Dirichlet noise is added to the prior probabilities $P(s, a)$ in the root node s_0 for each MCTS simulation. Specifically, $P(s, a) = (1 - \varepsilon)P_a + \varepsilon\eta_a$, where $\eta \sim \text{Dir}(0.03)$ and $\varepsilon = 0.25$.

During self-play, AlphaGo Zero uses a resign mechanism to avoid wasting computational resources on losing games. When the mean state-action value of the root node of the MCTS tree is lower than the threshold v_{resign} , the player choose to resign instead of continuing to play the game. AlphaGo Zero also considers the false positive rate of the resigned games, where the resign move is disabled for 10% of the self-play games, played until termination.

Eventually, the transition tuples (s_t, π_t, z_t) in each time step are collected as samples to train the neural network, where $z_t \in \{0, 1\}$ indicates whether the player who is placing the stone is winning.

2.4 Neural network modeling

2.4.1 Architecture

AlphaGo Zero uses a neural network architecture that employs convolutional neural networks (CNNs) to identify patterns in the board positions. The motivation is that patterns, invariant under rotation and reflection, are essential for effective decision-making in the games, and hence, we empower the reinforcement learning agent with the ability to extract important features by constantly updating its kernel.

Input. For the game of Go, the input features describing the position are structured as a 19×19 image, matched to the grid-structure of the board. In order to retain the Markov property, $N = 8$ history records of the board positions separated for each player, X_t and Y_t , are encoded as 16 feature planes used to represent the current player’s and opponent’s stones on the board. The planes representing the current player’s stones, carry a binary value of 1 if a stone is present and 0 if the intersection is empty or contains an opponent stone. Similarly, the other eight planes Y_t represent the opponent’s stones. Additionally, we add a final feature plane C representing the color to play. Such representation is stressed, as Go is not fully observable solely from the current stones, where repetitions are forbidden; similarly, the colour feature C is used to determine *komi* at the start of the game. Hence, the planes are concatenated into a single input in the following format:

$$s_t = [X_t, Y_t, X_{t-1}, Y_{t-1}, \dots, X_{t-7}, Y_{t-7}, C].$$

Output. Two outputs are determined by the neural network, i.e. the policy prior and the value of estimated winning probability, $(\mathbf{p}, v)$, is evaluated for each state s . The output $\mathbf{p}$ carries $19 \times 19 + 1$ values forming a probability distribution corresponding to the action probabilities for all actions, including the pass move; and v is a scalar in the range $[-1, 1]$ for the probability of winning for the current player.

Architecture. AlphaGo Zero uses a single residual neural network “body” (19 residual block), which is then passed into two separate “heads” for policy and value. The neural network architecture is shown in Figure 2. With the depth of the neural network, the residual blocks alleviate the performance degradation problem caused by vanishing and exploding gradients, which is then addressed by skip connections. This architecture allows AlphaGo Zero to process a complete board position as a single input and to identify complex patterns and features that are crucial for effective gameplay.

Training. The objective of the neural network is to learn a policy that maximizes the expected return given a action space. For AlphaGo Zero, the neural network is trained to maximize the similarity of the neural network’s move probabilities $\mathbf{p}$ to the MCTS search probabilities π using the cross-entropy loss, and to minimize the error between the predicted state value v and the game outcome z through MSE. Additionally, an L_2 -regularization term imposed on θ is used to prevent the neural network from overfitting. This results in the problem of minimizing the loss function ℓ defined by

$$\ell = (z - v)^2 + \pi^\top \log \mathbf{p} + c \|\theta\|^2.$$

Batches of 4096 transitions are randomly sampled from the recent 500,000 self-play games. The neural network is then optimized using the stochastic gradient descent (SGD) method with initial learning rate of 0.01, momentum of 0.9, and L_2 -regularization with $c = 10^{-4}$. Learning rate is set to decay using a staircase method, with decay rate of 0.1 during the course of training. This gradually adjust the parameters such that the optimization algorithm could rapidly converge to the better solution, and tackle problems such as oscillations and sluggish convergence. The new neural network is evaluated by the current best network at every 1,000 training step, by carrying out 400 games, and subsequently becomes the best player if the new player wins by a margin of $> 55\%$. This avoids noises from being selected.

3 Evaluation

3.1 Neural network architecture selection

Combined policy and value networks (“dual”) accompanied with residual network architecture (“res”) outperform the separated network architecture (“sep”) and the convolutional network architecture

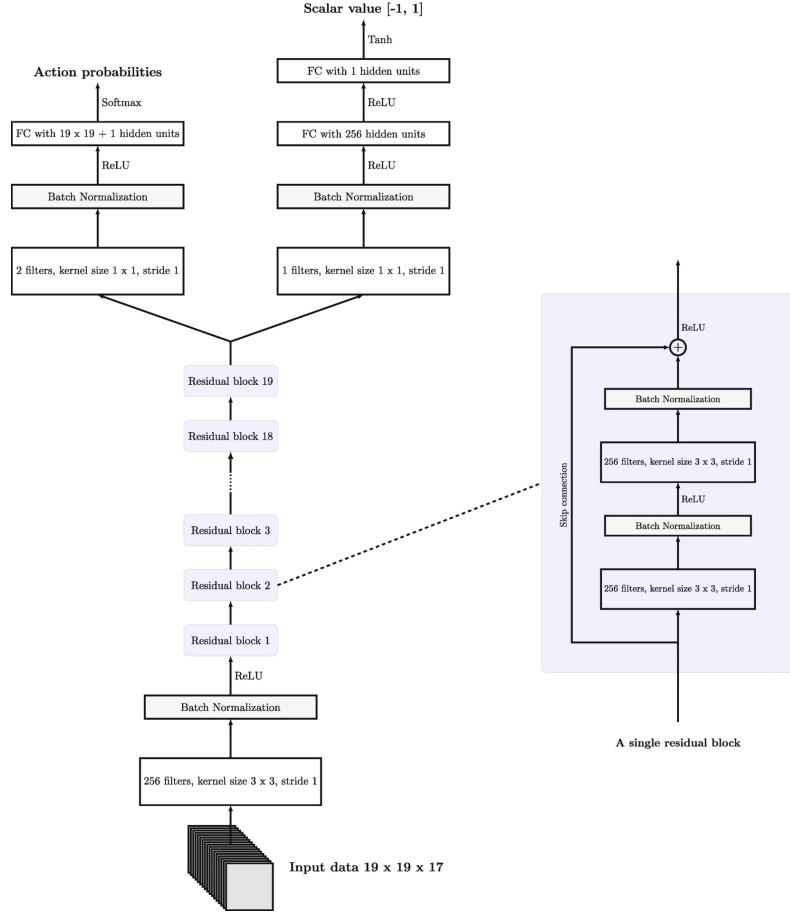


Figure 2: Neural network architecture for AlphaGo Zero.

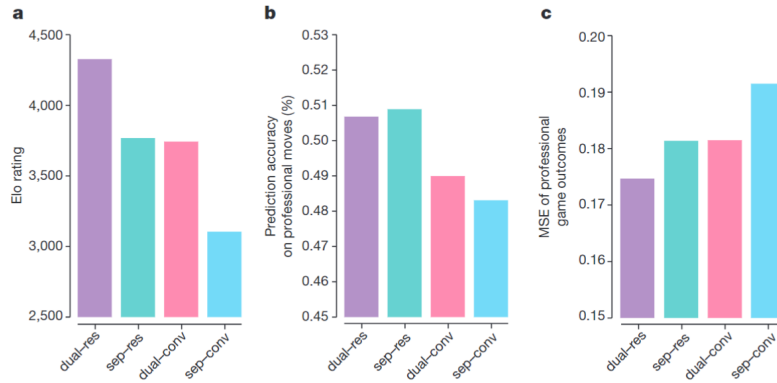


Figure 3: Comparison of neural network architectures in AlphaGo Zero.

(“conv”). The interpretation for this particular choice of AlphaGo Zero is that the dual objective regularizes the network to a common representation that supports multiple use cases, and improves computational efficiency. Comparison is shown in Figure 3(a).

3.2 Tournament against various opponent agents

To evaluate the performance of the AlphaGo Zero agent, one option is to let the agent play against various opponents, such as a professional human player or other AI agents, including itself with

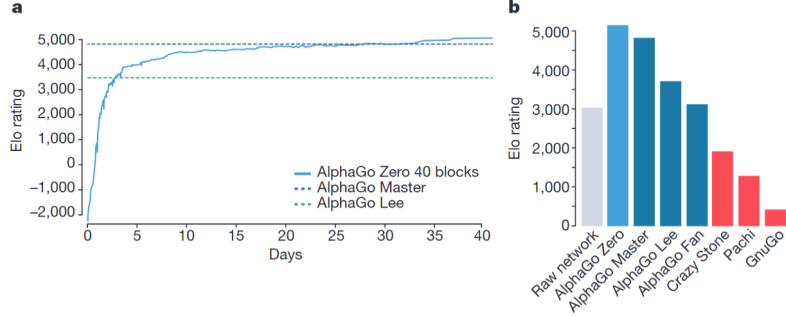


Figure 4: Performance of AlphaGo Zero resulting from the tournament among Go programs.

the weights of the neural network obtained from previous training iterations (see Figure 4). Then, we would be able to use the standard Elo rating to quantify its relative strength. The central assumption of Elo rating is that the performance of each player in each game is a normally distributed random variable. Furthermore, a player who wins the game is assumed to have performed at a higher level than the opponent for that game. By Bayesian logistic regression, we estimate the ratings $e(\cdot)$ computed by the *BayesElo* program using the standard constant $c_{elo} = 1/400$, and obtain an estimation of the probability that player a will defeat player b by a logistic function $p(a \text{ defeats } b) = (1 + 10^{c_{elo}(e(b) - e(a))})^{-1}$.

Eventually, the fully-trained AlphaGo Zero achieved a rating of 5,185, and surpassed AlphaGo Master by 89 games to 11 in a 100-game match with 2-hour time controls.

3.3 Expert Move Prediction

The second option is to use the trained neural network to predict moves and game outcomes from historical human gameplay. A second neural network using the same architecture is trained to predict expert moves, using the KGS Server dataset. Illustrated in Figures 3(b), 3(c), AlphaGo Zero achieved state-of-the-art prediction accuracy compared to the previous work. Although supervised learning achieved better initial performance, and was better at predicting human professional moves, the self-learned player performed much better overall.

It is also observed that AlphaGo Zero discovered a significant level of knowledge containing and beyond the level of human understanding. AlphaGo Zero rapidly progressed toward a sophisticated understanding of Go concepts, including *fuseki*, *tesuji*, *ko*, *yose*, etc.

4 Implications

AlphaGo Zero’s capability has soon been extended to master perfect information games like Chess and Shogi, other than Go, published in a 2018 paper by the same team with the name “AlphaZero” [2]. Later in 2020, MuZero was introduced with the generalized capability of mastering a wide range of games and tasks without access to any domain-specific information [3].

The empirical success of AlphaGo Zero has comprehensively demonstrate that a pure reinforcement learning approach for complex, high-dimensional problems is fully feasible, starting *tabula rasa* without any human examples or guidance. It also underscores the potential for AI to drive innovation and creativity. We have seen that the moves and strategies developed by AlphaGo Zero during its games against human opponents included novel and unconventional plays that had not been seen before, uncovering new patterns and challenging thinking. We will see more real-world decision making using AI in the near future.

The success of AI in recent years, including Alphago Zero and Chatgpt, also raises questions about ethical and societal implications of AI advancements. This serve as a reminder of the need for interdisciplinary collaboration, involving ethicists, policymakers, and stakeholders from different sectors, to take part in debate and address these challenges and harness the full potential of AI for the greater good of human. It is essential to ensure that AI are designed and used responsibly, with

mechanisms in place to address moral standard, protect privacy, and distribute the benefits equitably to the public.

References

- [1] Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T., Hui, F., Sifre, L., van den Driessche, G., Graepel T. & Hassabis, D. (2017) Mastering the game of Go without human knowledge. *Nature* **550**: 354-359. <https://www.nature.com/articles/nature24270>.
- [2] Silver, D. et al. (2018) A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science* **362**: 1140-1144. <https://www.science.org/doi/10.1126/science.aar6404>
- [3] Schrittwieser, J., Antonoglou, I., Hubert, T. et al. Mastering Atari, Go, chess and shogi by planning with a learned model. (2020) *Nature* **588**, 604–609. <https://doi.org/10.1038/s41586-020-03051-4>